Topik : ADT Queue

Tujuan Praktikum:

- Memahami konsep dan penggunaan ADT Queue
- Memahami Implementasinya dalam bahasa C

Petunjuk Praktikum:

1. Setiap ADT dibuat dengan format penamaan file sebagai berikut:
    a. Untuk file header : P<NIM><noSoal>.h
    b. Untuk file realisasi : P<NIM>noSoal>.c
    c. Untuk file driver : P<NIM>m<noSoal>.c

    contoh Mahasiswa dengan NIM 207006001 untuk soal no 1: P2070060011.h; P2070060011.c; P207006001m1.c
2. Untuk setip file yang dibuat, cantumkan header sebagai berikut:

```
/* NIM/Nama  :
   Nama file :
   Topik     :
   Tanggal   :
   Deskripsi :                                                    */
```

3. Seluruh file di-upload setelah dikompres menjadi 1 file dengan nama: P<nim>.zip.
4. HANYA PROGRAM YANG DAPAT DI-COMPILE YANG AKAN DIPERIKSA. File yang tidak dapat di-*compile* akan otomatis mendapatkan nilai 0.
5. Tugas ini bersifat INDIVIDUAL. Anda dipersilakan membaca dan menggunakan material yang Anda punya, tetapi tidak ada toleransi bagi pencontek. Jika terbukti, baik yang dicontek maupun yang mencontek akan mendapatkan nilai 0.
6. Ikuti petunjuk asisten untuk pengumpulan tugas ini.

QUEUE (ANTRIAN)

Pengertian Queue

QUEUE (Antrian) adalah list linier yang dikenali elemen pertama (HEAD) dan elemen terakhirnya (TAIL), aturan penyisipan dan penghapusan elemennya didefinisikan sebagai berikut:

- ✓ Penyisipan selalu dilakukan setelah elemen terakhir
- ✓ Penghapusan selalu dilakukan pada elemen pertama
- ✓ Satu elemen dengan yang lain dapat diakses melalui informasi NEXT

Struktur data ini banyak dipakai dalam informatika, misalnya untuk merepresentasi :

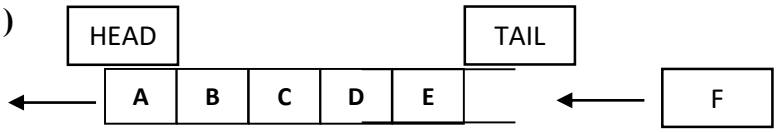
- ✓ Antrian job yang harus ditangani oleh sistem operasi
- ✓ Antrian dalam dunia nyata.

Maka secara lojik, sebuah QUEUE dapat digambarkan sebagai list linier yang setiap elemennya adalah :

```
type ElmtQ : < Info : InfoType, Next :address>
```

Dengan InfoType adalah sebuah type terdefinisi yang menentukan informasi yang disimpan pada setiap elemen queue, dan address adalah "alamat" dari elemen. Selain itu alamat elemen pertama (HEAD) dan elemen terakhir(TAIL) dicatat. Maka jika Q adalah Queue dan P adalah adaress, penulisan untuk Queue adalah : **Head(Q),Tail(Q), Info(Head(Q)), Info(Tail(Q))**

Queue (antrian) hampir mirip dengan stack, tapi hanya saja, data yang masuk pertama kali akan keluar pertama kali dari Queue. Bisa dilihat pada gambar dibawah ini **FIFO (First In First Out)**



Sama halnya dengan stack, Queue dapat disajikan baik dengan Array maupun dengan **struct**. Pada Array, stack ataupun queue yang disajikan bersifat statis. Ini disebabkan karena jumlah maksimal data pada array sudah ditentukan sejak awal.

Operasi pada Queue

Dalam penyajiannya queue, ada 2 proses yang terjadi, yaitu menambahkan elemen **(Enqueue)** dan mengurangi elemen **(Dequeue).** Seperti yang sudah dijelaskan bahwa array itu memiliki jumlah maksimal, maka pada proses Enqueue, perlu pengecekan apakah data yang di-enqueue melebihi jumlah maksimal array atau tidak.

Implementasi Antrian dengan Array

Seperti halnya pada tumpukan, maka dalam antrian kita juga mengenal ada dua operasi dasar, yaitu menambah elemen baru yang akan kita tempatkan di bagian belakang antrian dan menghapus elemen yang terletak di bagian depan antrian.

Disamping itu seringkali kita juga perlu melihat apakah antrian mempunyai isi atau dalam keadaan kosong. Operasi penambahan elemen baru selalu bisa kita lakukan karena tidak ada pembatasan banyaknya elemen dari suatu antrian. Tetapi untuk menghapus elemen, maka kita harus melihat apakah antrian dalam keadaan kosong atau tidak. Tentu saja kita tidak mungkin menghapus elemen dari suatu antrian yang sudah kosong.

```
/* File : queue.h */
/* deklarasi Queue yang diimplementasi dengan tabel kontigu */
/* HEAD dan TAIL adalah alamat elemen pertama dan terakhir  */
/* Queue mampu menampung MaxEl buah elemen */
#ifndef queue_H
#define queue_H
#include "boolean.h"
#include <stdlib.h>
#define Nil -1
/* Definisi elemen dan address */
typedef int infotype;
typedef int address;   /* indeks tabel */
/* Contoh deklarasi variabel bertype Queue : */
/*Versi I: tabel dinamik,Head dan Tail eksplisit */
/* tabel penyimpan elemen dipakai â€œmelingkarâ€, tidak ada pergeseran*/
typedef struct { infotype * T; /* tabel penyimpan elemen */
                   address HEAD;  /* alamat penghapusan */
                   address TAIL;  /* alamat penambahan */
                   int MaxEl;      /* MAx elemen queue */
                  } Queue;
/*Definisi Queue kosong: Head=Nil; TAIL=Nil.*/
/* Catatan implementasi: T[0] dipakai untuk menyimpan elemen pertama */
/* Queue pada pengisian elemen pertama kali setelah â€œkosongâ€ : tang
terisi adalah T[0]*/
/**** AKSES (Selektor)  *****/
/* Jika Q adalah Queue, maka akses elemen : */
#define Head(Q) (Q).HEAD
#define Tail(Q) (Q).TAIL
#define InfoHead(Q) (Q).T[(Q).HEAD]
#define InfoTail(Q) (Q).T[(Q).TAIL]
#define MaxEl(Q) (Q).MaxEl
/* Prototype */
boolean IsEmpty (Queue Q);
/* Mengirim true jika Q kosong: lihat definisi di atas */
boolean IsFull(Queue Q);
/*Mengirim true jika tabel penampung elemen Q sudah penuh */
/* yaitu mengandung MaxEl elemen*/
int NBElmt(Queue Q);
/* Mengirimkan banyaknya elemen queue. Mengirimkan 0 jika Q kosong */
/**** Kreator ***/
void CreateEmpty(Queue *Q, int Max);
/* I.S. sembarang */
/* F.S. Sebuah Q kosong terbentuk dan salah satu kondisi sbb: */
/* Jika alokasi berhasil, Tabel memori dialokasi berukuran Max */
/* atau : jika alokasi gagal, Q kosong dg Maksimum elemen=0 */
/* Proses : Melakukan alokasi,Membuat sebuah Q kosong */
/**** Destruktor ***/
void DeAlokasi(Queue *Q);
/* Proses: Mengembalikan memori Q */
/* I.S. Q pernah dialokasi dan tidak NULL */
/* F.S. Q menjadi tidak terdefinisi lagi, MaxEl(Q) diset 0 */
/**** Primitif Add/Delete ****/
void Add (Queue * Q, infotype X);
/* Proses: Menambahkan  X pada Q dengan aturan FIFO */
/* I.S. Q mungkin kosong, tabel penampung elemen Q TIDAK penuh */
/* F.S. X menjadi TAIL yang baru, TAIL "maju" . Suksesor dari (MaxEl-1)
adalah 0*/
/* Jika Tail(Q)=MaxEl-1 maka suksesornya adalah Head(Q)=0 */
void Del(Queue * Q, infotype* X);
```

```
/* Proses: Menghapus  X pada Q dengan aturan FIFO */
/* I.S. Q tidak mungkin kosong */
/* F.S. X = nilai elemen HEAD pd I.S.,HEAD "maju";Q mungkin kosong */
/* jangan lupa menyesuaikan nilai HEAD dan TAIL jika Queue menjadi kosong
*/
#endif
```

Soal . Simulasi antrian bank

Anda diminta untuk membuat sebuah program simulasi sederhana yang menggambarkan antrian pengunjung di suatu bank.

a. Ambilah ADT Queue dengan representasi Tabel Kontigu (yang di sertakan dalam file) dengan alokasi Memori dinamik. Queue akan digunakan untuk mensimulasikan antrian pengunjung bank yang akan dilayani. Setiap elemen queue menyatakan waktu kedatangan pengujung (integer > 0)
b. Tambahkan prosedur PrintQueue berikut ini ke ADT tersebut:

```
Procedure PrintQueue (input Q : Queue)
{ I.S. : Q terdefinisi, mungkin kosong }
{ F.S. : semua elemen Q tertulis di layar, dengan format :
         [ el-head ... el-tail], misal [4 8 12 9 10].
         Jika kosong, tampilan di layar adalah [ ]. }
```